

## Laboranweisung 11: Zeitabläufe simulieren & KI-Integration (VS Code)

Das heutige Ziel ist es, die Architektur unserer SimulationEngine (bzw. LogicEngine) vor dem Testat zu perfektionieren. Zudem liegt unser heutiger Fokus auf KI-Integration direkt in Ihrer Entwicklungsumgebung.

### Phase 1: Architektonisches Refactoring mit Copilot Chat

In unserem letzten Termin haben wir die 2-Phasen-Schleife (Reset → Evaluate → ClockTick) hartcodiert in die main.cpp geschrieben. Gleichzeitig existiert aber bereits eine LogicEngine, die Komponenten via `std::unique_ptr<Gate>` verwaltet. Momentan ruft `LogicEngine::doTick()` nur `evaluate()` auf. Das reicht aber für sequenzielle Logik nicht!

Aufgabe: Bauen Sie `doTick()` in der `LogicEngine.cpp` so um, dass sie den vollen 2-Phasen-Taktzyklus unterstützt.

Problem: Die `LogicEngine` hält abstrakte `Gate`-Pointer. Ein normales `Gate` kennt die Methode `onClockTick()` aber nicht! Wie finden wir heraus, welche Pointer im Vektor in Wahrheit ein `DFlipFlop` sind?



#### KI-Integration (VS Code Chat Panel):

Anstatt das selbst zu knobeln, nutzen wir die Chat-Funktion in VS Code!

1. Markieren Sie den Code der Funktion `doTick()` in `LogicEngine.cpp`.
2. Öffnen Sie den Copilot Chat (Chat-Symbol in der Seitenleiste) und stellen Sie folgende Frage:

*"Mein Vektor circuit enthält `std::unique_ptr<Gate>`. Einige dieser Objekte sind in Wahrheit vom Typ `DFlipFlop`, welches die zusätzliche Methode `onClockTick()` besitzt. Wie kann ich in dieser Funktion sicher über den Vektor iterieren und `onClockTick()` NUR auf den Objekten aufrufen, die wirklich ein `DFlipFlop` sind? Zeige mir die Lösung mit `dynamic_cast`."*

Umsetzung: Analysieren Sie den generierten Code, fügen Sie die fehlenden `#include`-Befehle (für `DFlipFlop.h`) hinzu und implementieren Sie die komplette 2-Phasen-Schleife innerhalb von `LogicEngine::doTick()`. Ihre `main.cpp` sollte danach die Taktung nur noch über `engine.doTick();` steuern!

### Phase 2: Das 4-Bit Schieberegister & Inline-Generierung

Wir bauen nun eine komplexe sequenzielle Schaltung: Das Schieberegister (Shift Register). Es besteht aus 4 hintereinandergeschalteten D-Flip-Flops. Bei jedem Takt rückt das Bit des Vorgängers einen Platz weiter nach rechts. (Aufbau: Schalter → DFF0 → DFF1 → DFF2 → DFF3).



#### KI-Integration (VS Code Inline Chat / Autocomplete):

Das händische Instanzieren und Verkabeln von 4 Flip-Flops mit `connectInput()` ist fehleranfällig. Lassen Sie die KI das direkt im Editor schreiben!

1. Gehen Sie in Ihre `main.cpp` an die Stelle, wo Sie die Bauteile aufbauen.
2. Drücken Sie `Ctrl + I` (Windows/Linux) bzw. `Cmd + I` (Mac), um den Inline-Chat direkt im Codefeld zu öffnen.
3. Geben Sie folgenden Prompt ein und drücken Sie Enter:

*"Erstelle einen Switch namens 'DataIn' und 4 DFlipFlop-Objekte (dff0 bis dff3) mit `std::make_unique`. Verkable sie in Serie: DataIn geht in Input 0 von dff0. dff0 geht in Input 0 von dff1, usw. Füge danach alle 5 Bauteile mit `engine.addComponent(std::move(...))` zur Engine hinzu."*

Umsetzung & Test:

1. Akzeptieren Sie den Vorschlag, wenn er korrekt aussieht.
2. Setzen Sie den Switch auf `true` (1) und rufen Sie `engine.doTick()` auf.
3. Setzen Sie den Switch auf `false` (0) und rufen Sie 4-mal `engine.doTick()` auf.
4. Drucken Sie die Outputs der Flip-Flops. Wandert die "1" bei jedem Takt ein Bauteil weiter?

### Phase 3: ASCII Timing-Diagramm visualisieren

Die nackte Konsolenausgabe von `true/false` ist unübersichtlich. Hardware-Designer nutzen Timing-Diagramme (Waveforms).

#### KI-Integration (Feature Generation via Chat):

Wir lassen die KI eine optisch ansprechende Hilfsfunktion schreiben.

1. Öffnen Sie wieder den VS Code Chat in der Seitenleiste.
2. Nutzen Sie diesen Prompt:

*"Schreibe mir eine C++ Hilfsfunktion `void printWaveform(int tick, bool val0, bool val1, bool val2, bool val3)`. Sie soll den Zustand von 4 Bits als kleines ASCII-Diagramm in einer Zeile ausgeben. Nutze für Low (0) ein Unterstrich-Zeichen '\_' und für High (1) ein Balken-Zeichen '|'. Beispielausgabe: Tick 3 | \_ ^ \_ \_"*

Umsetzung:

Fügen Sie diese Funktion in Ihre `main.cpp` ein und rufen Sie sie nach jedem `engine.doTick()` auf, um den Zustand von `dff0` bis `dff3` zu visualisieren.

## **Zusatzaufgaben (Für Schnelle)**

### **Zusatzaufgabe 1: Der Johnson-Zähler (Möbius-Zähler)**

Sie haben bereits ein Schieberegister aus 4 D-Flip-Flops gebaut. Verwandeln Sie es mit nur einem zusätzlichen Gatter in einen selbstlaufenden Ringzähler!

- Aufbau: Trennen Sie den Switch ab. Fügen Sie ein NotGate hinzu.
- Verbinden Sie den Ausgang des letzten Flip-Flops (dff3) mit dem NotGate.
- Verbinden Sie den Ausgang des NotGate zurück in den Eingang des ersten Flip-Flops (dff0).
- Beobachtung: Wenn Sie nun eine Schleife mit 10 doTick()-Aufrufen starten, beobachten Sie Ihr Timing-Diagramm. Die Bits füllen sich nacheinander mit Einsen (0000, 1000, 1100, 1110, 1111) und leeren sich danach wieder (0111, 0011, ...). Ein klassisches Lauflicht!

### **Zusatzaufgabe 2: Code-Review durch KI**

Nutzen Sie die KI, um Ihre Architektur "testat-ready" zu machen.

- Öffnen Sie die Datei LogicEngine.cpp.
- Markieren Sie den gesamten Code, öffnen Sie den Inline-Chat (Ctrl + I) und schreiben Sie:

*`"/explain Gibt es in diesem Code potenzielle Memory Leaks, auch wenn ich std::unique_ptr verwende? Entspricht der Code modernem C++17 Standard?"`*

- Diskutieren Sie im Team die Antwort der KI. Verstehen Sie genau, warum das automatische Aufräumen des Vektors in C++ durch Smart Pointers sicher funktioniert.